Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

Course Code: 23ECSPE1AD

Submitted by

Mallika Gourish Hegde 1BM24EC100

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

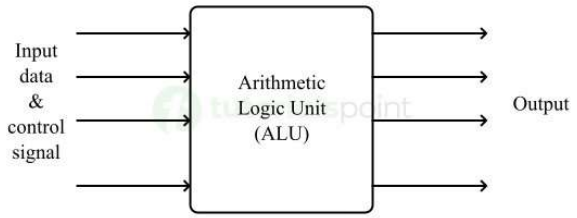
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026-2027

SL NO.	CONTENTS (DESIGN AND CODE)
1.	ARITHMETIC LOGIC UNIT
2.	COMPARATOR
3.	4-BIT PARITY CHECKER AND GENERATOR
4.	4-BIT BARREL SHIFTER
5.	TIMER MODULE
6.	RING COUNTER
7.	JOHNSON COUNTER
8.	SYNCHRONOUS COUNTER
9.	ASYNCHROMOUS COUNTER
10.	UNIVERSAL SHIFT REGISTER
11.	DIVIDE BY ‘N’ COUNTER
12.	MEALY TO DETECT 101
13.	MOORE TO DETECT 101
14.	RANDOM SEQUENCE GENERATOR(LFSR)

1. Arithmetic logic unit
   - Design logic :



   - Verilog code :

```
module alu_3bit (
    input [2:0] A,          // Input operand A
    input [2:0] B,          // Input operand B
    input [2:0] ALU_Sel,    // Select Line
    output reg [2:0] ALU_Out, // 3-bit Output
    output reg Carry_Out,   // Carry/Borrow
    Flag
    output reg Zero         // Zero Flag
    );
    always @(*) begin
     // Default assignments to prevent latches
     ALU_Out = 3'b000;
     Carry_Out = 1'b0;
      case (ALU_Sel)
        3'b000: begin // Addition: A + B
            {Carry_Out, ALU_Out} = A + B;
        end
        3'b001: begin // Subtraction: A - B
            {Carry_Out, ALU_Out} = A - B;
        end
        3'b010: begin // Bitwise AND: A & B
            ALU_Out = A & B;
        end
        3'b011: begin // Bitwise OR: A | B
            ALU_Out = A | B;
        end
        3'b100: begin // Bitwise XOR: A ^ B
         ALU_Out = A ^ B;
        end
        3'b101: begin // Logical Shift Left
         ALU_Out = A << 1;
        end
        3'b110: begin // Logical Shift Right
         ALU_Out = A >> 1;
        end
        3'b111: begin // Equality Check
         ALU_Out = (A == B) ? 3'b001 : 3'b000;
        end
        default: begin
            ALU_Out = 3'b000;
            Carry_Out = 1'b0;
        end
      endcase

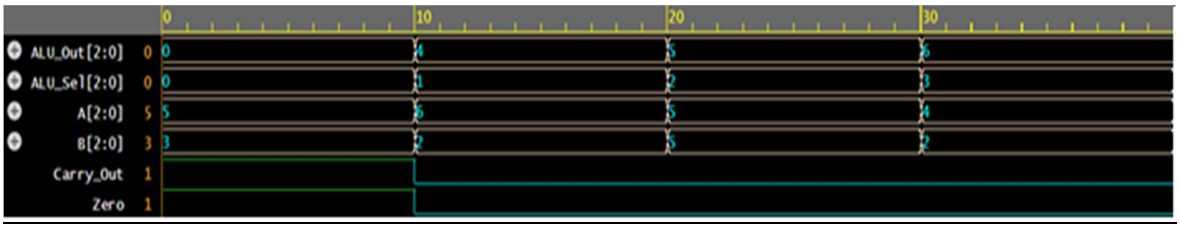
      // Zero flag goes high if the output is 0
      Zero = (ALU_Out == 3'b000) ? 1'b1 : 1'b0;
      end
endmodule
```

- Test bench :

```
module tb_alu_3bit;
 reg [2:0] A;
 reg [2:0] B;
 reg [2:0] ALU_Sel;
 wire [2:0] ALU_Out;
 wire Carry_Out;
 wire Zero;
 alu_3bit uut ( .A(A), .B(B), .ALU_Sel(ALU_Sel),
.ALU_Out(ALU_Out), .Carry_Out(Carry_Out),
.Zero(Zero) );

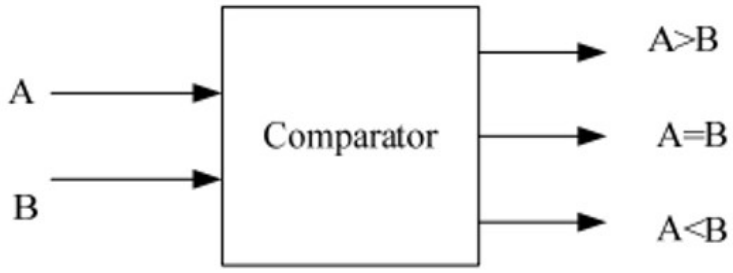
initial begin
$dumpfile("alu_3bit.vcd");
$dumpvars(0, tb_alu_3bit);
$monitor("Time=%0t | Sel=%b | A=%b | B=%b |
Out=%b | Carry=%b | Zero=%b", $time,
ALU_Sel, A, B, ALU_Out, Carry_Out, Zero);
 A = 3'b101; B = 3'b011; ALU_Sel = 3'b000; #10;
 A = 3'b110; B = 3'b010; ALU_Sel = 3'b001; #10;
 A = 3'b101; B = 3'b101; ALU_Sel = 3'b010; #10;
 A = 3'b100; B = 3'b010; ALU_Sel = 3'b011; #10;
 A = 3'b011; B = 3'b000; ALU_Sel = 3'b101; #10;
 A = 3'b110; B = 3'b110; ALU_Sel = 3'b111; #10;
 $finish;
 end
endmodule
```

- Result :



2. Comparator :

- Design logic :



- Verilog code :

```
module comparator_3bit (
  input [2:0] A ;
  input [2:0] B ;
  output A_greater, // High if A > B
  output A_less, // High if A < B
  output A_equal // High if A == B );
```

```
 // Continuous assignments for combinational comparison

   assign A_greater = (A > B);
   assign A_less = (A < B);
   assign A_equal = (A == B);
endmodule
```

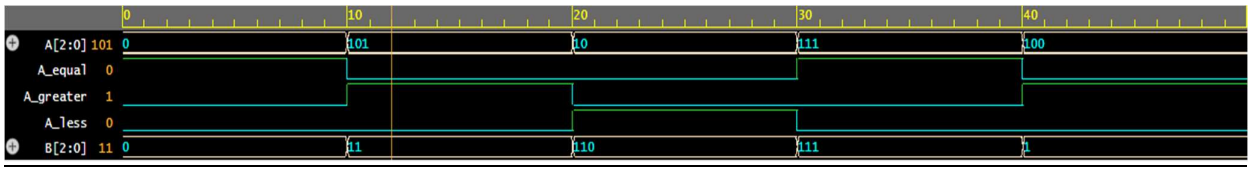
- Test bench :

```
module tb_comparator_3bit; // Inputs
 reg [2:0] A;
 reg [2:0] B; // Outputs
 wire A_greater;
 wire A_less;
 wire A_equal; // Instantiate the Unit Under Test
comparator_3bit uut (.A(A),
.B(B),.A_greater(A_greater), .A_less(A_less),
.A_equal(A_equal) );

initial begin
$dumpfile("comparator_3bit.vcd");
$dumpvars(0, tb_comparator_3bit);
```

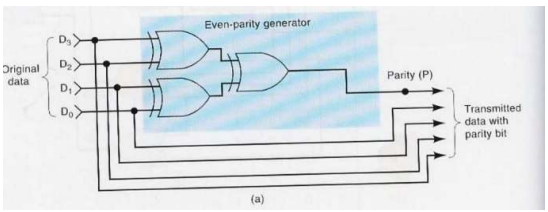
```
$monitor("Time=%0t | A=%b (%0d) | B=%b (%0d) | A>B=%b | A<B=%b | A==B=%b", $time, A, A, B, B, A_greater, A_less, A_equal);
   A = 3'b000; B = 3'b000; #10;
   A = 3'b101; B = 3'b011; #10;
   A = 3'b010; B = 3'b110; #10;
   A = 3'b111; B = 3'b111; #10;
   A = 3'b100; B = 3'b001; #10;
   $finish;
   end
endmodule
```

- Result :



3. 4-bit Parity Generator and Checker :

- Design logic :



- Verilog code :

```
module parity_generator (
  input [3:0] data_in, // 4-bit input data
  output [4:0] framed_out // 5-bit output (Parity
//bit + 4-bit data) );
// Even parity calculation using XOR reduction
// If the number of 1s is odd, parity_bit = 1. If
//even, parity_bit = 0.
  wire parity_bit;
  assign parity_bit = ^data_in;
  assign framed_out = {parity_bit, data_in};
endmodule
```

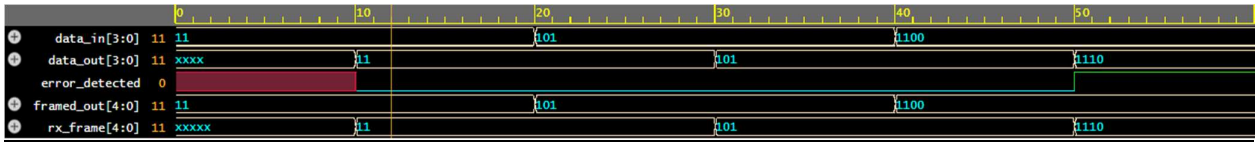
```
module parity_checker (
  input [4:0] rx_frame, // 5-bit received frame
  output [4:0] data_out,
  output error_detected // High if a transmission
//error is found );
  assign data_out = rx_frame[3:0];
  assign error_detected = ^rx_frame;
endmodule
```

- Test bench :

```
module tb_parity_system;
  reg [3:0] data_in;
  wire [4:0] framed_out;
  wire [3:0] data_out;
  wire error_detected; // Variable to simulate a
noisy transmission channel (error injection) reg
[4:0] rx_frame_noisy; // Instantiate Generator
parity_generator gen ( .data_in(data_in),
.framed_out(framed_out) );
// Instantiate Checker with the noisy channel
output
parity_checker chk ( .rx_frame(rx_frame_noisy),
.data_out(data_out),
.error_detected(error_detected) );
initial begin
$dumpfile("parity_system.vcd");
$dumpvars(0, tb_parity_system);
```

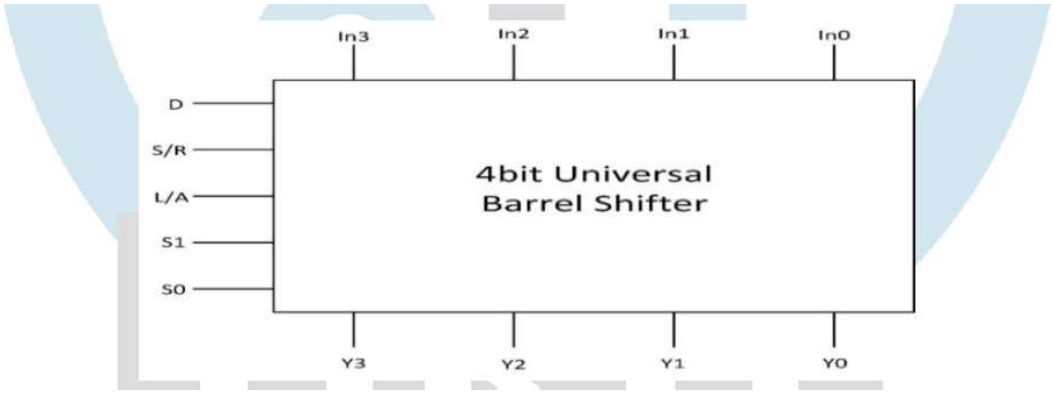
```
$monitor("Time=%0t | Data_In=%b |
Framed(Tx)=%b | Rx_Noisy=%b |
Error_Flag=%b", $time, data_in, framed_out,
rx_frame_noisy, error_detected);
  data_in = 4'b0011; #10;
  rx_frame_noisy = framed_out;  #10;
  data_in = 4'b0101; #10;
  rx_frame_noisy = framed_out; #10;
  rx_frame data_in = 4'b1100; #10;
  rx_frame_noisy = framed_out ^ 5'b00010; #10;
  $finish;
 end
endmodule
```

- Result :



4. 4-bit barrel shifter :

- Design logic :



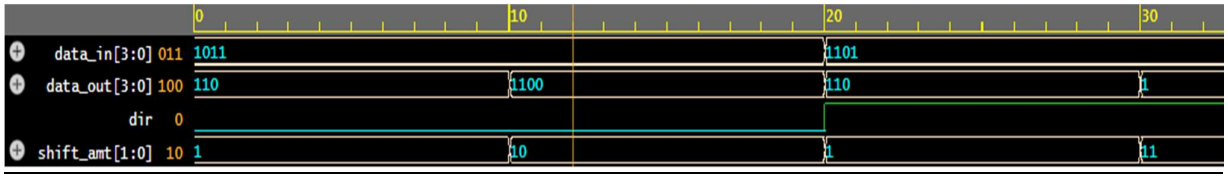
- Verilog code :

```
module barrel_shifter_4bit (
  input [3:0] data_in; // 4-bit input data
  input [1:0] shift_amt; // 2-bit shift amount
  input dir, // 0: Shift Left, 1: Shift Right
 output reg [3:0] data_out // 4-bit shifted output
);
 always @(*) begin
  if (dir == 1'b0) begin // Logical Shift Left
    case (shift_amt)
     2'b00: data_out = data_in;
     2'b01: data_out = {data_in[2:0], 1'b0};
     2'b10: data_out = {data_in[1:0], 2'b00};
     2'b11: data_out = {data_in[0], 3'b000};
     default: data_out = data_in;
    endcase
   end
else begin // Logical Shift Right
    case (shift_amt)
     2'b00: data_out = data_in;
     2'b01: data_out = {1'b0, data_in[3:1]};
     2'b10: data_out = {2'b00, data_in[3:2]};
     2'b11: data_out = {3'b000, data_in[3]};
     default: data_out = data_in;
    endcase
  end
 end
endmodule
```

- Test bench :

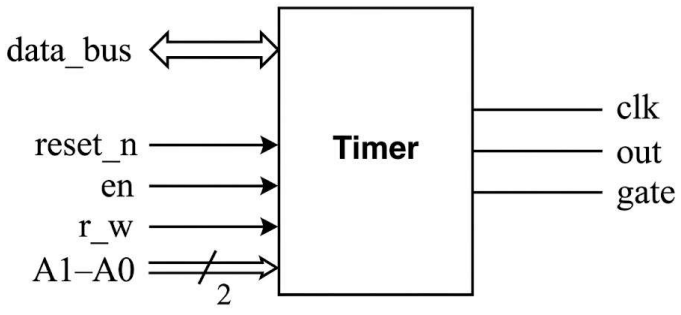
```
module tb_barrel_shifter_4bit; // Inputs
  reg [3:0] data_in;
  reg [1:0] shift_amt; reg dir; // Outputs
  wire [3:0] data_out;
 barrel_shifter_4bit uut ( .data_in(data_in),
.shift_amt(shift_amt), .dir(dir), .data_out(data_out)
);
initial begin
$dumpfile("barrel_shifter.vcd");
$dumpvars(0, tb_barrel_shifter_4bit);
$monitor("Time=%0t | Dir(0=L,1=R)=%b |
Data_In=%b | Shift_Amt=%b | Data_Out=%b",
$time, dir, data_in, shift_amt, data_out);
  data_in = 4'b1011;shift_amt = 2'b01; dir = 1'b0;
#10;
  data_in = 4'b1011; shift_amt = 2'b10; dir = 1'b0;
#10;
  data_in = 4'b1101; shift_amt = 2'b01; dir = 1'b1;
#10;
  data_in = 4'b1101; shift_amt = 2'b11; dir = 1'b1;
#10;
  $finish;
  end
endmodule
```

- Result :



5. Timer Module :

- Design logic :



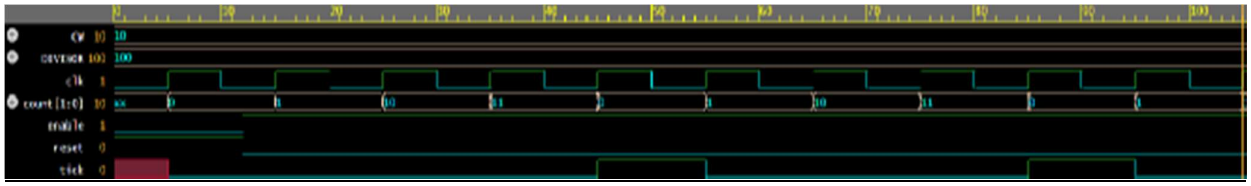
- Verilog code :

```
module timer_module (
      parameter WIDTH = 4)
(
    input clk,; // System Clock
    input rst_n ;// Active-low AsynchronousReset
    input start; // Enable/Start counting signal
    input load; // Load preset value signal
    input [WIDTH-1:0] preset_val,
    output reg timeout;
    output reg [WIDTH-1:0] count_out
);
   always @(posedge clk or negedge rst_n)
   begin
      if (!rst_n) begin
          count_out <= {WIDTH{1'b0}};
          timeout   <= 1'b0;
       end
    else if (load) begin
         count_out <= preset_val;
          timeout   <= 1'b0
      end
   else if (start) begin
    if (count_out > {WIDTH{1'b0}}) begin
      count_out <= count_out - 1'b1;
    if (count_out == 3'b001 || count_out == 1'b1)
      timeout <= 1'b1;
    else begin
      timeout <= 1'b0;
      end
    end
   else begin
     timeout <= 1'b1;
    end
   end
endmodule
```

- Test bench :

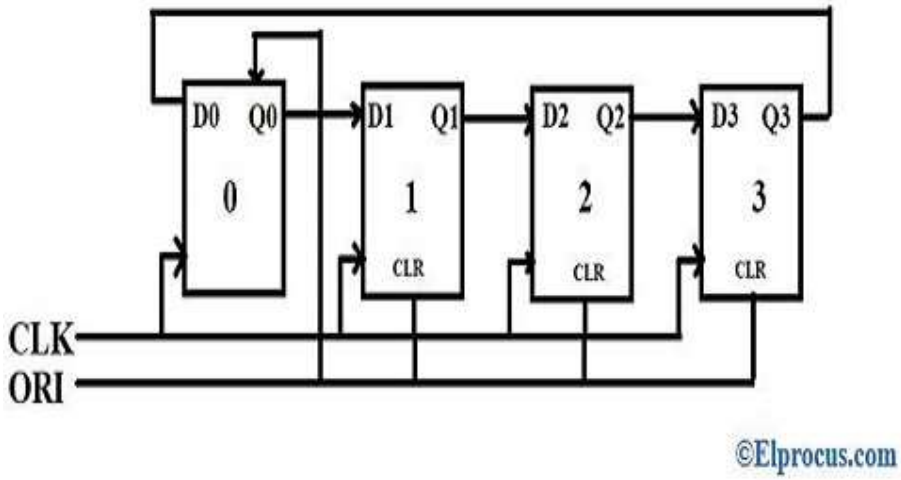
```
module tb_timer_module;
  parameter WIDTH = 4;
  reg clk;
  reg rst_n;
  reg start;
  reg load;
  reg [WIDTH-1:0] preset_val;
  wire timeout;
   wire [WIDTH-1:0] count_out;
   timer_module (.WIDTH(WIDTH)) uut (
.clk(clk),     .rst_n(rst_n), .start(start), .load(load),
.preset_val(preset_val), .timeout(timeout),
.count_out(count_out) );
always #5 clk = ~clk;
initial begin
   clk = 0; rst_n = 0; start = 0; load = 0;
   preset_val = 4'd0; #12;
   rst_n = 1; #10; // Load a preset value of 5
   preset_val = 4'd5;
   load = 1; #10;
   load = 0;
   countdown start = 1; #60;
   start = 0; #10;
    rst_n = 0; #10;
   $finish;
  end
endmodule
```

- Result :



6. Ring Counter :

- Design logic :



- Verilog code :

```
module ring_counter (
parameter WIDTH = 4)
( input clk;
  input rst_n;
  input load,
  input [WIDTH-1:0] seed;
  output reg [WIDTH-1:0] q_out;
```

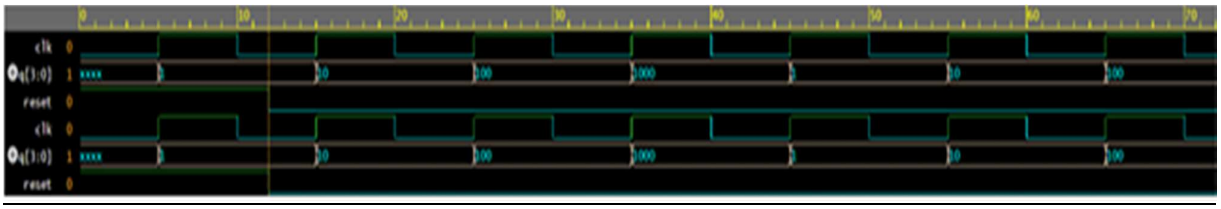
```
 always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
       q_out <= { {WIDTH-1{1'b0}}, 1'b1 };
     end
    else if (load) begin
       q_out <= seed;
    end
    else begin
       q_out <= {q_out[WIDTH-2:0],
q_out[WIDTH-1]};
    end
endmodule
```

- Test bench :

```
module tb_ring_counter;
parameter WIDTH = 4;
reg clk;
reg rst_n;
reg load;
reg [WIDTH-1:0] seed;
wire [WIDTH-1:0] q_out;
ring_counter #(.WIDTH(WIDTH)) uut ( .clk(clk),
.rst_n(rst_n), .load(load), .seed(seed),
.q_out(q_out) );
always #5 clk = ~clk;
initial begin
$dumpfile("ring_counter.vcd");
$dumpvars(0, tb_ring_counter);
```

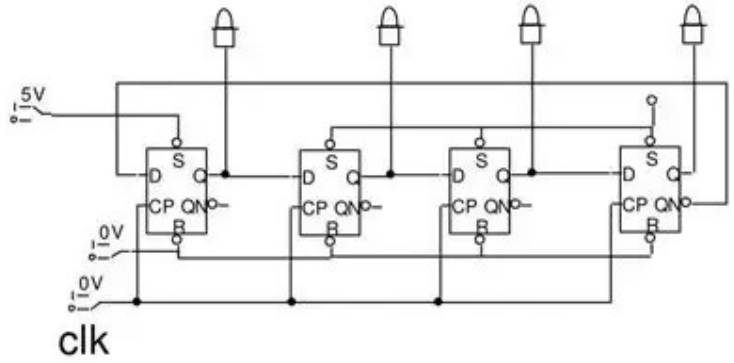
```
$monitor("Time=%0t | Rst=%b | Load=%b |
Q_Out=%b", $time, rst_n, load, q_out);
clk = 0; rst_n = 0; load = 0; seed = 4'b0001; #12;
 rst_n = 1; #40;
pattern seed = 4'b0100;
load = 1; #10;
load = 0; #30;
$finish;
end
endmodule
```

- Result :



7. Jhonson counter :

- Design logic :



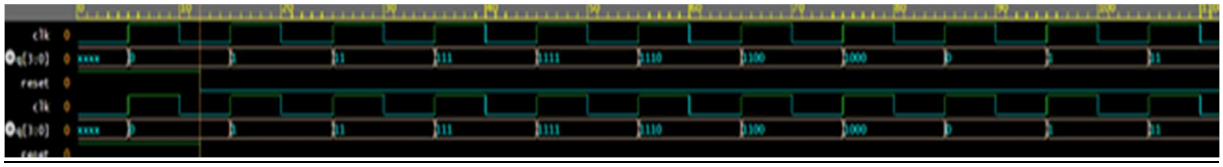
- Verilog code :

```
module johnson_counter
( parameter WIDTH = 4 )
(
  input clk,
  input rst_n,
  output reg [WIDTH-1:0] q_out );
always @(posedge clk or negedge rst_n)
 begin if (!rst_n) begin
    q_out <= {WIDTH{1'b0}};
  end
 else begin
   q_out <= {q_out[WIDTH-2:0],
~q_out[WIDTH-1]};
  end
 end
endmodule
```

- Test bench :

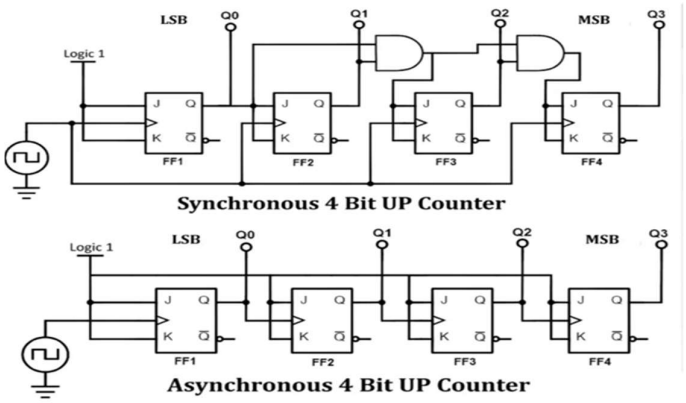
```
module tb_johnson_counter;
parameter WIDTH = 4;
reg clk;
reg rst_n;
wire [WIDTH-1:0] q_out;
johnson_counter #(.WIDTH(WIDTH)) uut (
.clk(clk), .rst_n(rst_n), .q_out(q_out) );
always #5 clk = ~clk;
initial begin
$dumpfile("johnson_counter.vcd");
$monitor("Time=%0t | Rst=%b | Q_Out=%b",
$time, rst_n, q_out);
clk = 0; rst_n = 0; #12;
rst_n = 1; #160;
$finish;
end
endmodule
```

- Result :



8. Synchronous and Asynchronous counter :

- Design logic :



- Verilog code Synchronous counter:

```
module async_ripple_counter_4bit (
  input clk,
  input rst_n,
 output reg [3:0] q);
 always @(posedge clk or negedge rst_n) begin
  if (!rst_n) q[0] <= 1'b0;
 else q[0] <= ~q[0]; end
```

```
 always @(negedge q[0] or negedge rst_n) begin
    if (!rst_n) q[1] <= 1'b0;
    else q[1] <= ~q[1]; end
 always @(negedge q[1] or negedge rst_n) begin
    if (!rst_n) q[2] <= 1'b0; else q[2] <= ~q[2];
end
 always @(negedge q[2] or negedge rst_n) begin
    if (!rst_n) q[3] <= 1'b0; else q[3] <= ~q[3];
end endmodule
```

- Verilog code for Asynchronous counter:

```
module sync_counter_4bit (
  input clk,
  input rst_n,
  input enable,
 output reg [3:0] q // 4-bit synchronous output );
always @(posedge clk or negedge rst_n) begin
```

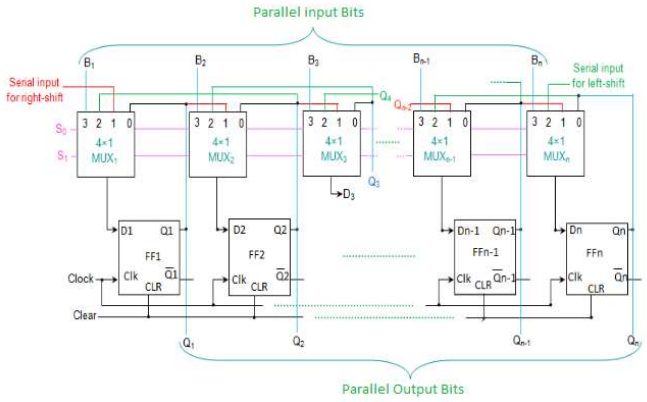
```
  if (!rst_n) begin
     q <= 4'b0000;
  end
 else if (enable) begin
     q <= q + 1'b1;
  end
 end
endmodule
```

- Result :



9. Shift register :

- Design logic :



- Verilog code

```
module universal_shift_register
( parameter WIDTH = 4 )
(
  input clk,
  input rst_n,
  input [1:0] sel,
  input serial_in_right,
  input serial_in_left,
  input [WIDTH-1:0] parallel_in,
  output reg [WIDTH-1:0] q_out);
 always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
   q_out <= {WIDTH{1'b0}};
  end
```

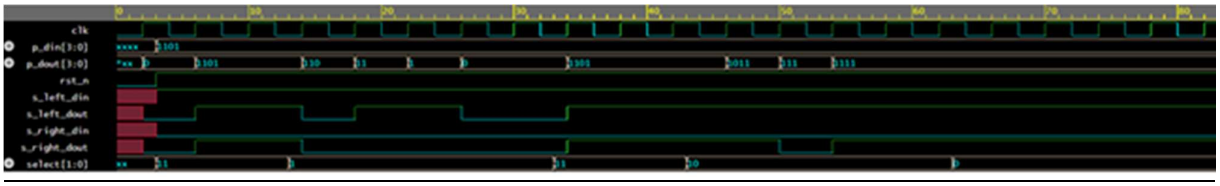
```
else begin
    case (sel)
    2'b00: q_out <= q_out;
    2'b01: q_out <= {serial_in_right,
q_out[WIDTH-1:1]};
   2'b10: q_out <= {q_out[WIDTH-2:0],
serial_in_left};
   2'b11: q_out <= parallel_in;
  default: q_out <= q_out;
   endcase
  end
  end
endmodule
```

- Test bench :

```
module tb_universal_shift_register;
parameter WIDTH = 4;
reg clk;
reg rst_n;
reg [1:0] sel;
reg serial_in_right;
 reg serial_in_left;
 reg [WIDTH-1:0] parallel_in;
wire [WIDTH-1:0] q_out;
universal_shift_register #(.WIDTH(WIDTH)) uut
( .clk(clk), .rst_n(rst_n), .sel(sel),
.serial_in_right(serial_in_right),
.serial_in_left(serial_in_left),
.parallel_in(parallel_in), .q_out(q_out) );
always #5 clk = ~clk;
```

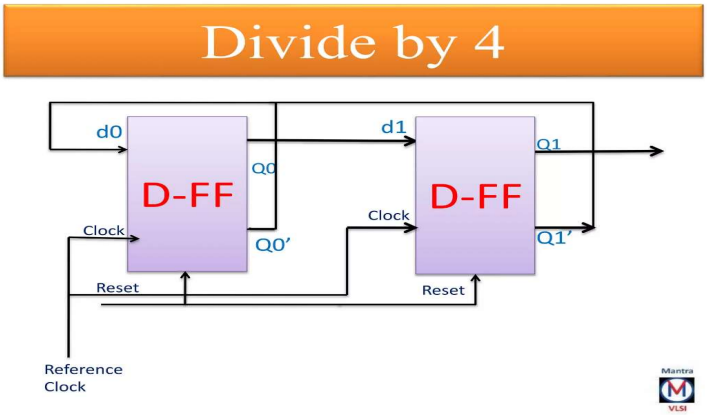
```
initial begin
$dumpfile("shift_reg.vcd");
$dumpvars(0, tb_universal_shift_register);
$monitor("Time=%0t | Sel=%b | P_In=%b |
S_Right=%b | S_Left=%b | Q_Out=%b", $time,
sel, parallel_in, serial_in_right, serial_in_left,
q_out);
clk = 0; rst_n = 0; sel = 2'b00; serial_in_right = 0;
serial_in_left = 0; parallel_in = 4'b0000; #12;
 rst_n = 1;
sel = 2'b11; parallel_in = 4'b1010; #10;
sel = 2'b01; serial_in_right = 1'b1; #30;
sel = 2'b00; #20;
sel = 2'b10; serial_in_left = 1'b0; #30;
 $finish;
end
endmodule
```

- Result :



10. Devide by –N

- Design logic :



- Verilog code ‘

```
module divide_by_n
( parameter N = 4)
(
  input clk,
 input rst_n,
 output reg clk_out);
 reg [$clog2(N)-1:0] count;
```

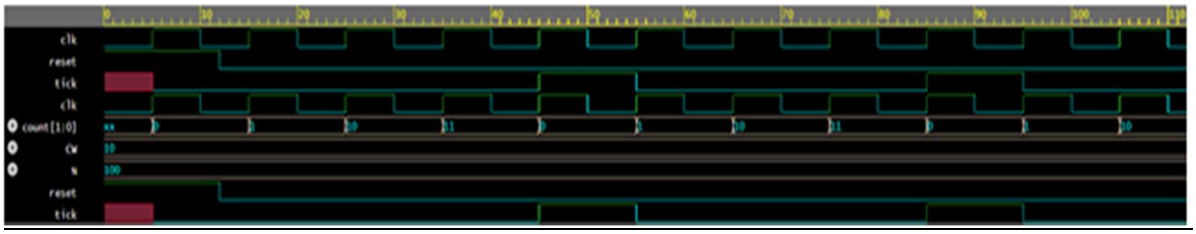
```
always @(posedge clk or negedge rst_n) begin
   if (!rst_n) begin
     count <= { $clog2(N) {1'b0} };
     clk_out <= 1'b0;
  end
  else if (count == (N/2 - 1)) begin
   count <= { $clog2(N) {1'b0} };
   clk_out <= ~clk_out;
  end
 end
endmodule
```

- Test bench :

```
module tb_divide_by_n;
parameter N = 4;
reg clk;
reg rst_n;
wire clk_out;
divide_by_n #(.N(N)) uut ( .clk(clk), .rst_n(rst_n),
.clk_out(clk_out) );
always #5 clk = ~clk;
```

```
initial begin
$dumpfile("div_n.vcd");
$dumpvars(0, tb_divide_by_n);
$monitor("Time=%0t | Master_Clk=%b |
Div_Clk_Out=%b", $time, clk, clk_out);
clk = 0; rst_n = 0; #12; rst_n = 1; #120;
 $finish;
end
endmodule
```

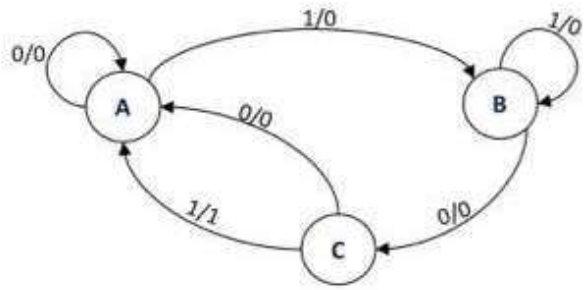
- Result :



11. Mealy and More FSM :

i. Mealy FSM Example: "101" Sequence Detector

- Design logic :



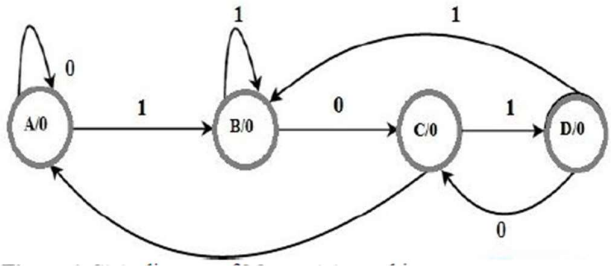
- Verilog code :

```
module mealy_seq_detector (
    input clk,
    input rst_n,
    input din,
    output reg det );
 parameter S0 = 2'b00;
 parameter S1 = 2'b01;
 parameter S2 = 2'b10;
 reg [1:0] current_state, next_state
 always @(posedge clk or negedge rst_n) begin
  if (!rst_n)
        current_state <= S0;
 else
       current_state <= next_state; end
```

```
  always @(*) begin
    case (current_state)
      S0: next_state = din ? S1 : S0;
      S1: next_state = din ? S1 : S2;
      S2: next_state = din ? S1 : S0;
      default: next_state = S0;
    endcase
  end
 always @(*) begin
  case (current_state)
   S0: det = 1'b0;
   S1: det = 1'b0;
   S2: det = (din == 1'b1) ? 1'b1 : 1'b0;
   default: det = 1'b0;
  endcase
end
endmodule
```

ii. Moore FSM Example: "101" Sequence Detector

- Design logic :



- Verilog code :

```
module moore_seq_detector (
    input clk,
    input rst_n,
    input din,
    output reg det );
 parameter S0 = 2'b00;
 parameter S1 = 2'b01
 parameter S2 = 2'b10;
 parameter S3 = 2'b11;
 reg [1:0] current_state, next_state;
 always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        current_state <= S0;
   else
        current_state <= next_state;
 end
```

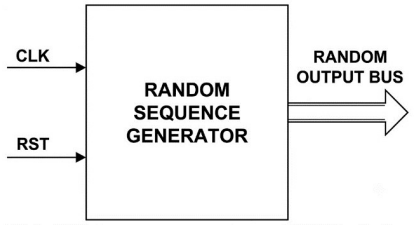
```
 always @(*) begin
    case (current_state)
        S0: next_state = din ? S1 : S0;
        S1: next_state = din ? S1 : S2;
        S2: next_state = din ? S3 : S0;
        S3: next_state = din ? S1 : S2;
       default: next_state = S0;
    endcase
 end
 always @(*) begin
     det = (current_state == S3) ? 1'b1 : 1'b0;
 end
endmodule
```

- Test bench :

```
module tb_seq_detector;
 reg clk;
 reg rst_n;
 reg din;
 wire det;
moore_seq_detector uut ( .clk(clk), .rst_n(rst_n),
.din(din), .det(det)
always #5 clk = ~clk;
initial begin
$dumpfile("moore_fsm.vcd");
$dumpvars(0, tb_moore_seq_detector);
$monitor("Time=%0t | Rst=%b | Din=%b |
Det_Out=%b", $time, rst_n, din, det);
clk = 0; rst_n = 0; din = 0; #12;
rst_n = 1;
@(posedge clk); din = 1'b1;
@(posedge clk); din = 1'b0;
@(posedge clk); din = 1'b1;
@(posedge clk); din = 1'b0;
@(posedge clk); din = 1'b1;
@(posedge clk); din = 1'b0;
@(posedge clk); din = 1'b0;
@(posedge clk); din = 1'b1;
@(posedge clk); din = 1'b0;
@(posedge clk);
$finish;
end
endmodule
```

12. Random sequence generator :

- Design logic :



- Verilog code :

```
module lfsr_4bit (
   input clk,
   input rst_n,
   input load,
   input [3:0] seed,
   output reg [3:0] q_out );
   wire feedback;
  assign feedback = q_out[3] ^ q_out[2];
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        q_out <= 4'b0001;
   end
   else if (load) begin
        q_out <= (seed == 4'b0000) ? 4'b0001 :
seed;
   end
   else begin
        q_out <= {q_out[2:0], feedback};
  endmodule
```

- Test bench :

```
module tb_lfsr_4bit;
reg clk;
reg rst_n;
reg load;
reg [3:0] seed;
wire [3:0] q_out;
 lfsr_4bit uut ( .clk(clk), .rst_n(rst_n), .load(load),
.seed(seed), .q_out(q_out) );
always #5 clk = ~clk;
initial begin
$dumpfile("lfsr_4bit.vcd");
$dumpvars(0, tb_lfsr_4bit);
$monitor("Time=%0t | Rst=%b | Load=%b |
Random_State=%b (Decimal: %0d)", $time, rst_n,
load, q_out, q_out);
clk = 0; rst_n = 0; load = 0; seed = 4'b0000; #12;
 rst_n = 1; #160;
seed = 4'b1011; load = 1; #10;
load = 0; #60;
$finish;
end
endmodule
```